

ROTEIRO DE ESTUDO AUTÔNOMO

Da Recursividade ao Merge Sort e Quick Sort

Execute cada etapa antes de consultar a conferência.

Objetivo: acompanhar a execução de Merge Sort e Quick Sort, reconhecer suas chamadas recursivas e prever como cada algoritmo transforma uma lista até chegar à ordenação.

Etapa 1 — Retomando a recursividade

Considere:

```
def contagem(n):  
    if n == 0:  
        return  
  
    print(n)  
    contagem(n - 1)
```

Para `contagem(4)`, as chamadas são:

```
contagem(4)  
↓  
contagem(3)  
↓  
contagem(2)  
↓  
contagem(1)  
↓  
contagem(0)
```

Complete:

```
contagem(6)  
  
6  
↓  
—  
↓  
4  
↓  
—  
↓  
2  
↓  
—  
↓  
0
```

Conferência: 5, 3, 1.

O problema diminui segundo $n \rightarrow n - 1$.

Etapa 2 — E se o problema diminuir pela metade?

```
def dividir(n):  
    if n <= 1:  
        return  
  
    dividir(n // 2)
```

Começando com $n = 16$, complete:

```
16  
↓  
—  
↓  
4  
↓  
—  
↓  
1
```

Conferência: 8 e 2.

Agora faça para 32:

32 → ____ → ____ → ____ → ____ → 1

Conferência: 16 → 8 → 4 → 2.

$n \rightarrow n-1 \rightarrow n-2 \rightarrow n-3 \rightarrow \dots$

versus

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

A segunda sequência chega ao caso-base muito mais rapidamente. Essa ideia já apareceu na Busca Binária.

Etapa 3 — Busca Binária como recursão

[3, 8, 12, 17, 25, 31, 42, 56, 68]

Queremos encontrar 56. O elemento central é 25. Como $56 > 25$, qual região ainda pode conter 56?

A) [3, 8, 12, 17]

B) [31, 42, 56, 68]

Conferência: B.

Na região [31, 42, 56, 68] temos novamente o mesmo tipo de problema: procurar 56 em uma lista ordenada menor.

BUSCAR

↓

BUSCAR em um problema menor

↓

BUSCAR em um problema menor

Etapa 4 — Uma mudança importante

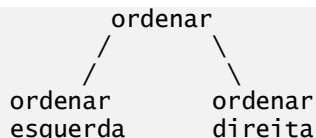
Até aqui, a cada chamada escolhemos um problema menor. Agora considere:

[8, 3, 7, 4]

Divida ao meio:

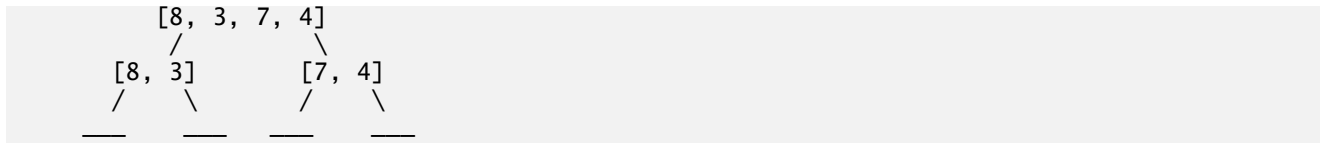
[8, 3] [7, 4]

Para ordenar toda a lista, precisamos ordenar as duas partes. Surge uma estrutura com duas chamadas:



Esta é a ponte entre Recursividade e Dividir e Conquistar.

Etapa 5 — Continue dividindo



Conferência: [8] [3] [7] [4].

É necessário continuar dividindo [8]? ☐ Sim ☐ Não

Conferência: Não. Uma lista com um único elemento já está ordenada. Esse é o caso-base.

Etapa 6 — O primeiro problema

Depois da divisão temos [8] [3] [7] [4], mas queremos [3, 4, 7, 8]. Dividir não é suficiente. Precisamos reconstruir as listas em ordem.

No Merge Sort, essa reconstrução utiliza a operação MERGE (intercalação).

Etapa 7 — Aprendendo a operação MERGE

Considere duas listas já ordenadas:

[3, 8] [4, 7]

Compare 3×4 . O menor é 3 $\rightarrow$ resultado = [3].

Agora 8×4 . Complete:

resultado = [3, ____]

Conferência: 4.

Agora 8×7 :

resultado = [3, 4, ____]

Conferência: 7.

Sobrou 8. Resultado final:

[3, 4, 7, 8]

Regra do merge: compare os menores elementos ainda disponíveis das duas listas e transfira o menor para o resultado.

Etapa 8 — Treine somente o Merge

[2, 9] [4, 6]

$2 \times 4 \rightarrow$ escolhe ____

$9 \times 4 \rightarrow$ escolhe ____

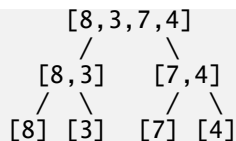
$9 \times 6 \rightarrow$ escolhe ____

restante $\rightarrow$ ____

resultado = [____, ____, ____, ____]

Conferência: $2 \rightarrow 4 \rightarrow 6 \rightarrow 9$; resultado $[2, 4, 6, 9]$.

Etapa 9 — Juntando recursividade + Merge



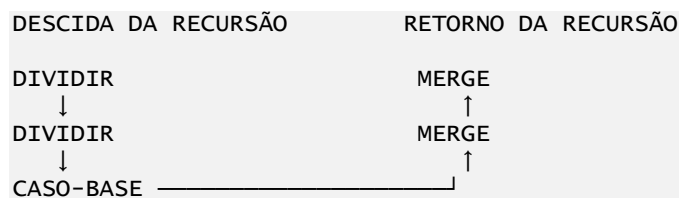
Chegamos ao caso-base. Agora começa o retorno das chamadas recursivas.

```
merge([8], [3]) → [____]
merge([7], [4]) → [____]
```

Conferência: $[3, 8]$ e $[4, 7]$.

```
merge([3, 8], [4, 7])
→ [____, ____, ____, ____]
```

Conferência: $[3, 4, 7, 8]$.



Etapa 10 — Transformando o MERGE manual em código

Até aqui, você executou a intercalação manualmente. Agora vamos transformar exatamente esse procedimento na função `merge()`. Ela recebe duas listas que já estão ordenadas e produz uma única lista ordenada.

A função `merge()` NÃO é recursiva. A recursividade está em `merge_sort()`. O `merge()` combina os resultados durante o retorno das chamadas recursivas.

Precisamos acompanhar uma posição em cada lista. Para isso, usamos dois índices independentes:

```
i → posição atual na lista esquerda
j → posição atual na lista direita
```

Considere:

```
esquerda = [3, 8]
direita  = [4, 7]

i = 0
j = 0
```

No início, `esquerda[i]` vale 3 e `direita[j]` vale 4. Como $3 \leq 4$, copiamos 3 para resultado e avançamos apenas `i`.

```
resultado = [3]
i = 1
j = 0
```

Agora `esquerda[i]` vale 8 e `direita[j]` vale 4. Como 4 é menor, copiamos 4 e avançamos apenas `j`.

```
resultado = [3, 4]
i = 1
j = 1
```

Compare novamente: 8×7 . Copiamos 7 e avançamos j.

```
resultado = [3, 4, 7]
i = 1
j = 2
```

Agora $j == \text{len}(\text{direita})$. A lista direita terminou. Não precisamos mais comparar: basta acrescentar o que restou da esquerda.

```
resultado.extend(esquerda[i:])
resultado = [3, 4, 7, 8]
```

Complete o rastreamento antes de conferir:

```
merge([2, 9], [4, 6])

i = 0, j = 0
2 × 4 → copia ____ ; índice que avança: ____
9 × 4 → copia ____ ; índice que avança: ____
9 × 6 → copia ____ ; índice que avança: ____
lista que terminou: _____
elemento restante: _____
resultado: [____, ____, ____, ____]
```

Conferência: copia 2 e avança i; copia 4 e avança j; copia 6 e avança j; a lista direita termina; resta 9; resultado [2, 4, 6, 9].

Agora observe a implementação completa:

```
def merge(esquerda, direita):
    resultado = []

    i = 0
    j = 0

    # Enquanto ainda houver elementos nas duas listas
    while i < len(esquerda) and j < len(direita):

        # Copia o menor elemento disponível
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    # Uma das listas terminou.
    # Acrescenta os elementos restantes da outra.
    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])

    return resultado
```

Associe cada trecho à operação manual:

- A. $i = 0$ e $j = 0$
- B. $\text{while } i < \text{len}(\text{esquerda}) \text{ and } j < \text{len}(\text{direita})$
- C. $\text{if } \text{esquerda}[i] \leq \text{direita}[j]$
- D. $i += 1$ ou $j += 1$
- E. $\text{resultado.extend}(\dots)$

1. Avançar o cursor da lista de onde um elemento foi retirado.
2. Iniciar os dois cursores.
3. Continuar comparando enquanto as duas listas ainda possuem elementos.
4. Acrescentar o restante quando uma das listas terminar.
5. Decidir qual dos dois elementos disponíveis deve ser copiado.

Conferência: A → 2; B → 3; C → 5; D → 1; E → 4.

Agora o fluxo completo está definido: merge_sort() divide recursivamente; merge() intercala as partes ordenadas durante o retorno.

Etapa 11 — Integrando merge() e merge_sort()

Agora observe como a função recursiva merge_sort() utiliza a função merge() que você acabou de estudar. O primeiro procedimento divide e resolve os subproblemas; o segundo combina as soluções.

```
def merge_sort(lista):  
    if len(lista) <= 1:  
        return lista  
  
    meio = len(lista) // 2  
  
    esquerda = lista[:meio]  
    direita = lista[meio:]  
  
    esquerda = merge_sort(esquerda)  
    direita = merge_sort(direita)  
  
    return merge(esquerda, direita)
```

Associe: A. Caso-base B. Divisão C. Recursão D. Combinação

1. return merge(esquerda, direita)
2. if len(lista) <= 1:
3. esquerda = merge_sort(esquerda)
 direita = merge_sort(direita)
4. esquerda = lista[:meio]
 direita = lista[meio:]

Conferência: A → 2; B → 4; C → 3; D → 1.

Etapa 12 — Execute um Merge Sort completo

[6, 2, 8, 3, 1, 7, 4, 5]

Primeira divisão:

[6,2,8,3] [1,7,4,5]

Complete as próximas divisões:

[6,2,8,3] \

_____/ \

```
[1,7,4,5]
  /      \
  _____
```

Continue até listas unitárias e somente então faça os merges.

Conferência: Resultado final: [1,2,3,4,5,6,7,8]. Se chegou diretamente ao resultado, repita registrando os estados intermediários.

Etapa 13 — Agora mude a estratégia

Considere novamente [8, 3, 7, 4]. No Merge Sort, a divisão ocorreu pela posição. Agora use o último valor como referência:

pivô = 4

Valor	< 4	= 4	> 4
8			X
3			
7			
4			

Conferência: 8 → maior; 3 → menor; 7 → maior; 4 → igual.

```
menores = [3]
pivô     = [4]
maiores  = [8,7]
```

Etapa 14 — O que o pivô conseguiu fazer?

```
todos < 4      todos > 4
  ↓             ↓
  [3]          4      [8,7]
```

O valor 4 já separou os dados em duas regiões. Essa operação é chamada de particionamento.

O particionamento é o princípio central do Quick Sort.

Etapa 15 — O problema reaparece

Ainda precisamos ordenar [8,7]. Use novamente o último elemento como pivô.

```
pivô = ____

menores que 7 = ____
iguais a 7    = ____
maiores que 7 = ____
```

Conferência: pivô = 7; menores = []; iguais = [7]; maiores = [8].

[] e [8] já atingiram o caso-base.

```
quick_sort([8,7])
= [] + [7] + [8]
= [7,8]
```

```
[3] + [4] + [7,8]
= [3,4,7,8]
```

Você acabou de executar Quick Sort.

Etapa 16 — Compare o mecanismo

MERGE SORT

```
  [8,3,7,4]
 /   \
[8,3]  [7,4]
```

QUICK SORT

```
  [8,3,7,4]
    |
  pivô 4
 /   \
[3]    [8,7]
```

Merge Sort divide principalmente pela: [] relação com pivô [] posição

Conferência: posição.

Quick Sort divide principalmente pela: [] relação dos valores com pivô [] posição central

Conferência: relação dos valores com um pivô.

Etapa 17 — Descubra o código do Quick Sort

```
def quick_sort(lista):
    if len(lista) <= 1:
        return lista

    pivo = lista[-1]

    menores = []
    iguais = []
    maiores = []

    for elemento in lista:
        if elemento < pivo:
            menores.append(elemento)
        elif elemento == pivo:
            iguais.append(elemento)
        else:
            maiores.append(elemento)

    return (
        quick_sort(menores)
        + iguais
        + quick_sort(maiores)
    )
```

Associe: A. Caso-base B. Escolha do pivô C. Particionamento D. Recursão

```
1. pivo = lista[-1]
2. if len(lista) <= 1:
3. menores / iguais / maiores
4. quick_sort(menores) e quick_sort(maiores)
```

Conferência: A → 2; B → 1; C → 3; D → 4.

Etapa 18 — Execute Quick Sort sozinho

[6, 2, 8, 3, 5]

pivô = ____

menores = _____

iguais = _____

maiores = _____

Depois aplique exatamente a mesma operação sobre cada partição com mais de um elemento.

Conferência: Resultado final: [2,3,5,6,8].

Etapa 19 — Algo estranho pode acontecer no Quick Sort

[1,2,3,4,5,6,7,8]

Use sempre o último elemento como pivô.

1ª chamada:

pivô = 8

menores = [1,2,3,4,5,6,7]

maiores = []

2ª chamada:

pivô = ____

menores = _____

maiores = ____

Conferência: pivô = 7; menores = [1,2,3,4,5,6]; maiores = [].

Próximo pivô: ____

Conferência: 6.

8 elementos

|

7 elementos

|

6 elementos

|

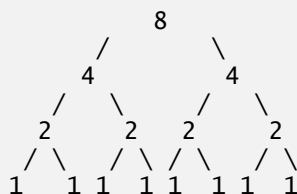
5 elementos

|

...

Etapa 20 — Compare com Merge Sort

Merge Sort:



Quick Sort do exemplo:

8
|

```

7
|
6
|
5
|
4
|
...

```

Qual árvore possui menor profundidade? [] Merge Sort [] Quick Sort deste exemplo

Conferência: Merge Sort.

Etapa 21 — Conectando com complexidade

```

n
↓
n/2
↓
n/4
↓
n/8
↓
...

```

Uma divisão aproximadamente pela metade possui cerca de $\log_2 n$ níveis. No Merge Sort, cada nível processa aproximadamente n elementos nas intercalações.

Merge Sort: $n \times \log n \rightarrow O(n \log n)$

Etapa 22 — Quick Sort equilibrado

```

      n
     / \
    n/2 n/2
   / \ / \
  n/4 n/4 n/4 n/4

```

Quando o pivô gera partições aproximadamente equilibradas, a profundidade também fica próxima de $\log n$.

Quick Sort — comportamento típico: $O(n \log n)$

Etapa 23 — Quick Sort desequilibrado

```

n
|
n-1
|
n-2
|
n-3
|
...

```

O trabalho se aproxima de $n + (n-1) + (n-2) + \dots + 1$, que cresce proporcionalmente a n^2 .

Quick Sort — pior caso: $O(n^2)$

Complete:

Merge Sort — pior caso: $O(\rule{1cm}{0.4pt})$

Quick Sort — caso típico: $O(\rule{1cm}{0.4pt})$

Quick Sort — pior caso: $O(\rule{1cm}{0.4pt})$

Conferência: $n \log n$; $n \log n$; n^2 .

Etapa 24 — Experimento com o pivô

[1, 2, 3, 4, 5, 6, 7, 8, 9]

Estratégia A: último elemento → pivô 9

[1, 2, 3, 4, 5, 6, 7, 8] 9 []

Estratégia B: neste exemplo, use pivô 5.

menores = _____

pivô = 5

maiores = _____

Conferência: [1,2,3,4] 5 [6,7,8,9].

Qual escolha produziu partições mais equilibradas? ☐ pivô 9 ☐ pivô 5

Conferência: pivô 5.

Etapa 25 — Cuidado com uma conclusão errada

[1, 2, 100, 3, 4]

O elemento da posição central é 100.

menores = [1,2,3,4]

pivô = 100

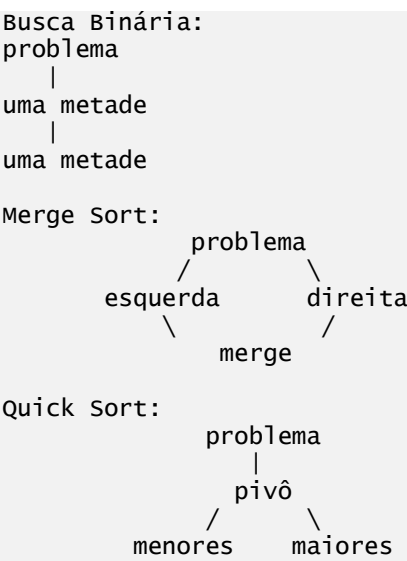
maiores = []

A divisão ficou equilibrada? ☐ Sim ☐ Não

Conferência: Não.

O balanceamento depende da distribuição dos valores em relação ao pivô, não simplesmente da posição original do elemento.

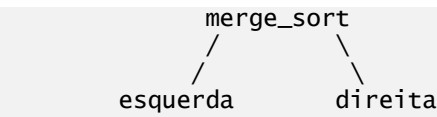
Etapa 26 — Retome a recursividade



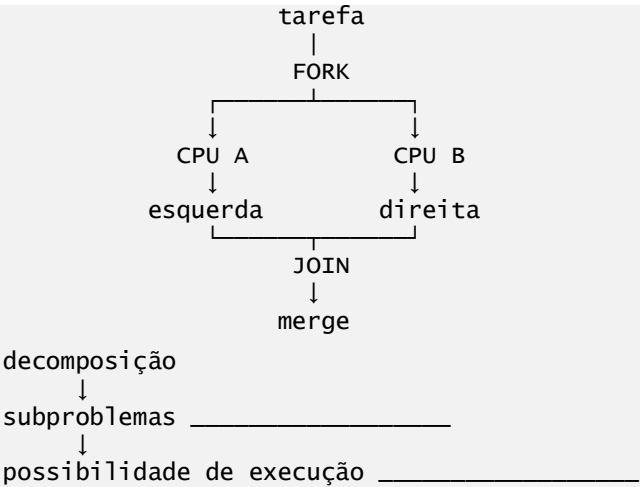
Algoritmo	Reduz o problema?	Duas chamadas?	Pivô?	Merge?
Busca Binária	Sim			
Merge Sort				
Quick Sort				

Conferência: Busca Binária: Sim/Não/Não/Não. Merge Sort: Sim/Sim/Não/Sim. Quick Sort: Sim/Sim/Sim/Não.

Etapa 27 — Recursividade e paralelismo



As duas ordenações precisam ser concluídas, mas cada uma pode realizar seu próprio trabalho sem conhecer o resultado da outra.



Conferência: independentes; paralela.

A recursividade não cria paralelismo automaticamente. A independência entre as tarefas é o requisito importante.

Etapa 28 — Teste de previsão

[7, 2, 6, 1]

Merge Sort — complete:

Primeira divisão:

Casos-base:

Primeiros merges:

Resultado:

Quick Sort — usando o último elemento como pivô:

pivô = ____

menores = _____

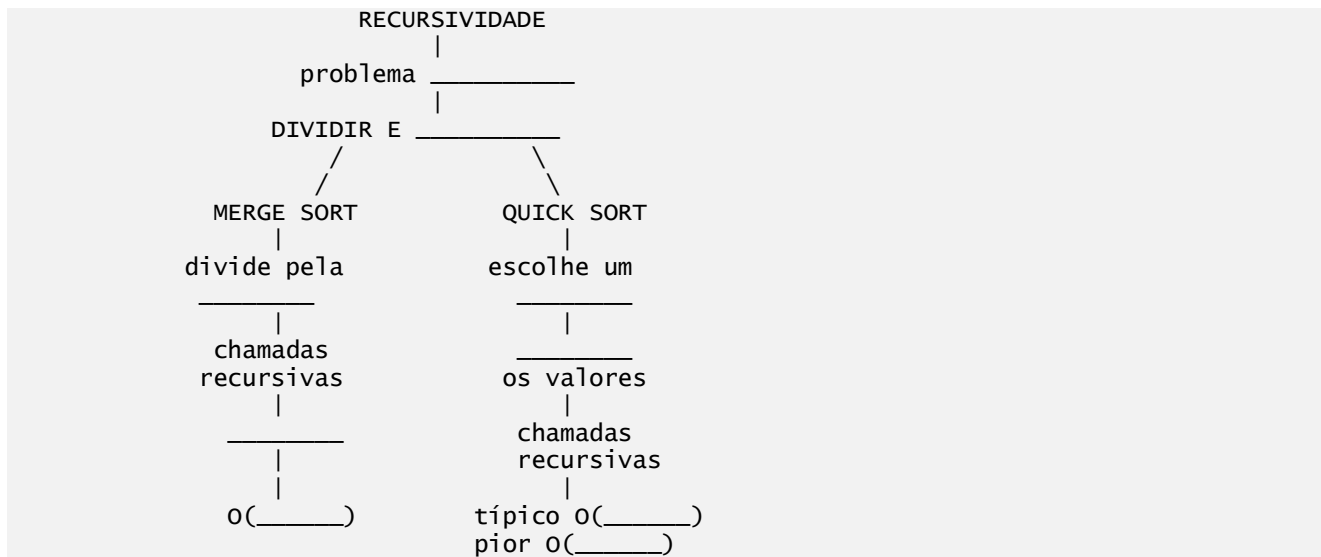
maiores = _____

Continue até o caso-base.

Resultado:

Conferência: Resultado final para ambos: [1,2,6,7]. Se algum estado intermediário divergir, volte à primeira divergência.

Etapa 29 — Mapa mental final



Conferência: problema menor; Dividir e Conquistar; posição; pivô; merge; particiona; $n \log n$; $n \log n$; n^2 .

Etapa 30 — Verificação final

Antes de encerrar, confirme se você consegue executar mentalmente cada uma destas operações:

- ☐ Dividir uma lista como o Merge Sort.
- ☐ Identificar o caso-base do Merge Sort.
- ☐ Fazer merge de duas listas já ordenadas.
- ☐ Reconstruir a lista durante o retorno da recursão.
- ☐ Particionar uma lista a partir de um pivô.
- ☐ Aplicar novamente Quick Sort às partições.
- ☐ Reconhecer uma árvore Quick Sort equilibrada ou degenerada.
- ☐ Prever quando a escolha do pivô poderá ser problemática.
- ☐ Associar Merge Sort a $O(n \log n)$.
- ☐ Associar Quick Sort a $O(n \log n)$ típico e $O(n^2)$ no pior caso.
- ☐ Reconhecer que recursividade é o mecanismo e Dividir e Conquistar é a estratégia.

**Recursividade → problema menor → dois subproblemas → Dividir e Conquistar → Merge Sort → Quick Sort
→ complexidade**